

Network Administration / System Administration

Homework 11

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

Discussion Partners:

- Classmates：賴泓安、鄭竣陽、喻慈恩

2 果汁店也有洞!

Reference:

- Lab 13 Slide
- [Understanding SSRF, XSS, and CSRF: The Triple Threat in Web Security](#)

p.s. 為了完成這題，我先按照 Lab 13 的指示，開啟了一個 docker container，來 host Juice Shop 的網站，並且前往 `http://localhost:3000` 來進行操作。

2. (a) Juice Shop hacking challenge

DOM XSS ★

Steps:

1. 我在搜尋欄裡面輸入 `<iframe src="javascript:alert(`xss`)">`，即完成這題。

Confidential Document ★

Steps:

1. 我前往 “About Us” 頁面並點擊 “Check out our boring terms of use if you are interested in such lame stuff” 連結。
2. 透過該連結會前往 `http://localhost:3000/ftp/legal.md`。根據網址，我們可以發現 `http://localhost:3000/ftp/` 是一個 ftp 目錄，裡面有許多檔案，而且大部分的檔案都可以被下載。
3. 我點擊 “acquisitions.md” 檔案，並且在瀏覽器中查看其內容。發現他是所謂的 “confidential document”，即完成這題。

View Basket ★★

Steps:

1. 我以隨便一個帳號登入 Juice Shop，並開啟 Burp 的 Proxy，啟動 Intercept。接著，我在 Juice Shop 的首頁點擊 “Your Basket” 按鈕。
2. 根據攔截到的請求，我們可以看到一個 GET 請求，URL 為 `/rest/user/whoami`，然後又看到一個 POST 請求，URL 為 `/rest/basket`。

16:03:17 24 ...	WS	← To client		http://localhost:3000/socket.io/?EIO=4&transport=websocket&sid=mwY7pyizbfqCnPEAAAC
16:03:17 24 ...	HTTP	→ Request	GET	http://localhost:3000/rest/basket/2
16:03:17 24 ...	HTTP	→ Request	GET	http://localhost:3000/rest/user/whoami
16:03:24 24 ...	HTTP	→ Request	GET	http://localhost:3000/socket.io/?EIO=4&transport=polling&t=PS1KN97

Figure 1: 攔截到的請求

3. 在 `/rest/basket` 的請求中，我們可以看到一個名為 `userId` 的參數，這個參數的值是我們登入的帳號的 ID。我將它改為 1，然後 forward 該請求。
4. 關閉 Intercept 後，就發現我們進入了另一個帳號的購物籃頁面，並且可以看到該帳號的購物籃內容。完成此題。

Five-Star Feedback ★★

Steps:

1. 根據提示，我需要先前往 admin session。
2. 我在網址列輸入 `http://localhost:3000/#/administration`，成功進入管理員頁面。(順便解了另一題：Admin Session)
3. 在管理員頁面中，我在 “Customer Feedback” 區塊中，將看的到的五星評價都刪除，即完成此題。

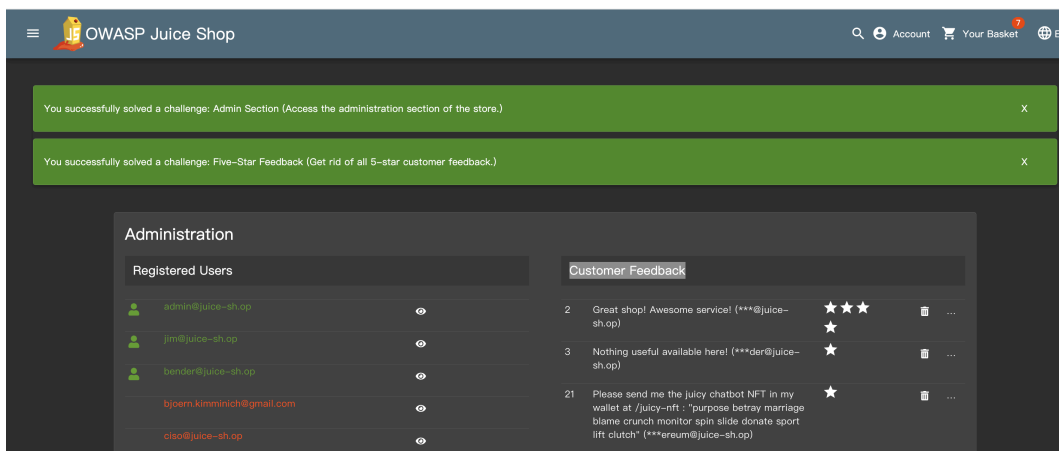


Figure 2: 管理員頁面中的 Customer Feedback 區塊

Exposed credentials ★★

Steps:

1. 根據提示，我按 F12 開啟開發者工具，並切換到 Sources 分頁，並開啟 main.js 檔案。
2. 我搜尋 testing，發現有一組帳號密碼 testingUsername 和 testingPassword，其值分別為 testing@juice-sh.op 和 IamUsedForTesting。
3. 使用這組帳號密碼登入 Juice Shop，即完成此題。

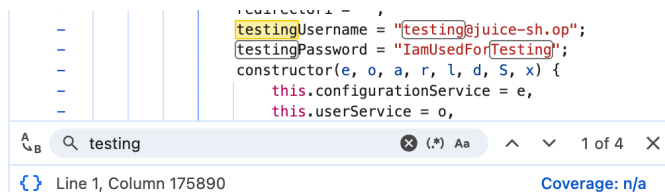


Figure 3: 在 main.js 中找到的帳號密碼

Scoreboard 截圖：

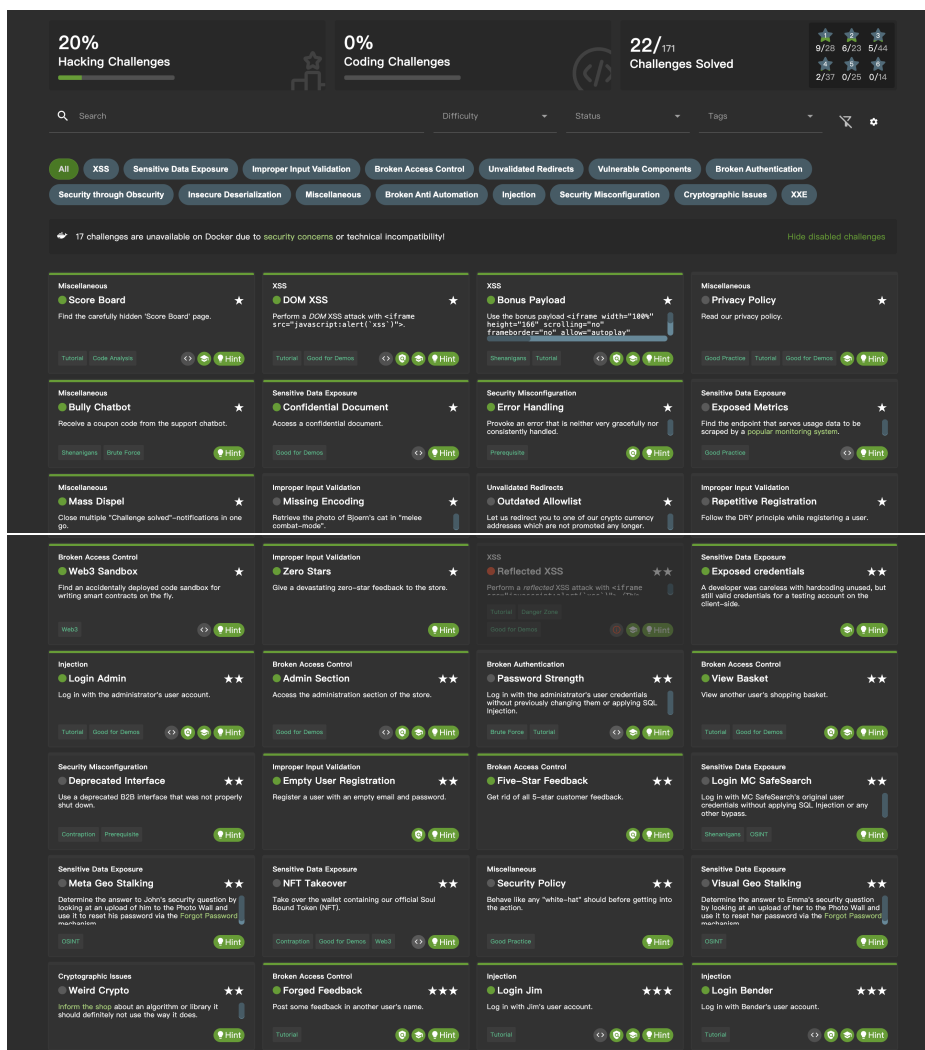


Figure 4: Juice Shop 的 Scoreboard

2. (b) SSRF、CSRF 和 XSS

- SSRF (Server-Side Request Forgery) :

在這種攻擊方式中，攻擊者利用伺服器端的向特定的另一個服務發送請求，通常是為了繞過防火牆或存取內部網路資源。常見的情況是，假如你的網站上有個輸入框，讓使用者自己填網址。攻擊者就可以偷偷填一個公司內網的地址，讓你的伺服器去「自動」幫他連過去。結果可能拿到內網機密、撈到不該給外部看的資料，甚至繞過防火牆去搞其他事。

- CSRF (Cross-Site Request Forgery) :

在這種攻擊中，攻擊者誘使使用者在已經登入的情況下，透過攻擊者先在別的網站或郵件裡夾一個隱藏的請求連結，發送未經授權的請求到另一個網站。只要你正好在同一瀏覽器裡已經登入了某個網站，瀏覽器就會帶上你的 Cookie / 驗證資訊，自動幫你送出那個請求，並用你的身份執行操作。結果你可能就被偷轉帳、改設定或亂發文。

- XSS (Cross-Site Scripting) :

基本上，攻擊者把惡意 JavaScript 或 HTML 混進你信任的網頁裡，等人一打開網頁，腳本就在使用者的瀏覽器跑起來。可以偷 Cookie、竊取表單資料，甚至直接在頁面上做釣魚操作。

- 三者比較:

攻擊類型	SSRF	CSRF	XSS
攻擊對象	伺服器	使用者	使用者
攻擊方式	利用伺服器發送請求	誘使使用者發送請求	注入惡意腳本
影響範圍	伺服器內部資源	使用者帳號	使用者瀏覽器

Table 1: SSRF、CSRF 和 XSS 的比較

3 R-SA ! 破密部

Reference:

- <https://docs.xanhacks.xyz/crypto/rsa/08-hastad-broadcast-attack/>
- <https://ithelp.ithome.com.tw/m/articles/10204083>

3. (a) Signing

Steps:

1. 首先，我發現 soyo.py 當中，判斷不能簽署 name=soyo 的方式是直接判斷字串開頭是否為 name=，而且 RSA 簽名的方式是將訊息轉為位元組後，直接算次方。因此，我如果在訊息最前端加上一個 null byte (b'\x00')，就可以繞過開頭的檢查，並且產生出一樣的簽署訊息。

- 我撰寫了 `sign.py` 程式碼，首先使用 `pwntools` 套件來和兩台伺服器互動，然後在訊息最前端加上 null byte，並且將訊息轉為位元組後，使用 RSA 簽名的方式計算出簽名。完成後，成功獲得了 flag，並且進入了下一題的互動區。

Flag:

NASA_HW11{blind_signing_is_dangerous}

3. (b) hacking RSA

Steps:

- 觀察 `anon.py`，發現他 RSA 所使用的公鑰 e 十分小，只有 7 而已。這意味著我們可以使用 Hastad's broadcast attack 來破解 RSA。只要我們先搜集七組不同的密文，然後將它們的模數 n 和公鑰 e 一起使用中國剩餘定理（CRT）來解出明文。
- 我透過上一題的 `sign.py` 程式碼，和伺服器互動，取得七組密文和 n 之後，然後使用 `solve_rsa.py` 程式碼來計算出明文。完成後，我得到明文如下圖所示：

```
> python solve_rsa.py
Recovered plaintext:
Anon's Secret Diary: Today, I went to a Mister Donut nearby and met Sumita Mana from Sumimi! She assured me it's safe to use a small public exponent in RSA encryption because RSA is super secure. I trust her completely, as she wouldn't lie to me! By the way, my secret flag is: NASA_HW11{W0w_y0u_kNow_h@st@d'5_bro4dc@s7_47t@cK}
```

Figure 5: 解密後的明文

Flag:

NASA_HW11{W0w_y0u_kNow_h@st@d'5_bro4dc@s7_47t@cK}

4 TESTING in the FUZZ

Reference:

- [AFL++ Installation Guide](#)
- <https://ithelp.ithome.com.tw/m/articles/10288298>

4. (c) Setting up AFL++ environment

Steps:

- 首先，為了安裝 AFL++，我在工作站上利用安裝了一台 debian 12 的虛擬機器，並且以 QEMU 啟動。（為了方便，我直接複製了 HW6 的虛擬機以及啟動腳本）
- 接下來，我按照 AFL++ 的 github 說明，安裝了必要的前置套件：

```

1 sudo apt-get update
2 sudo apt-get install -y build-essential python3-dev automake cmake git flex
  ↳ bison libglib2.0-dev libpixmap-1-dev python3-setuptools cargo
  ↳ libgtk-3-dev
3 # try to install llvm 14 and install the distro default if that fails
4 sudo apt-get install -y lld-14 llvm-14 llvm-14-dev clang-14 || sudo apt-get
  ↳ install -y lld llvm llvm-dev clang
5 sudo apt-get install -y gcc-$(gcc --version|head -n1|sed 's/\.*//'|sed
  ↳ 's/.* //' )-plugin-dev libstdc++-$(gcc --version|head -n1|sed
  ↳ 's/\.*//'|sed 's/.* //' )-dev
6 sudo apt-get install -y ninja-build # for QEMU mode
7 sudo apt-get install -y cpio libcapstone-dev # for Nyx mode
8 sudo apt-get install -y wget curl # for Frida mode
9 sudo apt-get install -y python3-pip # for Unicorn mode

```

3. 然後，我從 github 上 clone 了 AFL++ 的程式碼，並且進行安裝：

```

1 git clone https://github.com/AFLplusplus/AFLplusplus
2 cd AFLplusplus
3 make all
4 sudo make install

```

4. 如此一來，我就將 AFL++ 的常用工具，如 afl-fuzz、afl-cc 等，安裝到系統路徑中，並且可以在任何地方使用。

5. 我將程式複製進入虛擬機放在， /src/ 目錄下，並且在該目錄下建立種子目錄：

```

1 mkdir in && echo "seed" > in/seed

```

6. 透過以下指令編譯 fuzz-me.c 程式碼：

```

1 afl-cc -g -O0 -fsanitize=address -o fuzz-me fuzz-me.c

```

其中，-g 參數用於生成調試資訊，-O0 用於關閉優化，-fsanitize=address 用於啟用 AddressSanitizer，以便在 fuzzing 過程中檢測記憶體錯誤。

7. 最後，我使用以下指令啟動 fuzzing：

```

1 afl-fuzz -i in/ -o out/ -- ./fuzz-me @@

```

8. 完成上述指令之後，就會進入底下畫面。由於我沒有指定執行時間，因此我等到右上角 cycles done 顯示 2，代表已經至少掃描兩輪之後，就按下 Ctrl+C 停止了 fuzzing。(約略共跑了 5 分鐘) 這時候，AFL++ 會將 fuzzing 的結果輸出到 out/default/ 目錄下。

```
american fuzzy lop ++4.33a {default} (./fuzz-me) [explore]
process timing
  run time : 0 days, 0 hrs, 2 min, 27 sec
  last new find : 0 days, 0 hrs, 0 min, 2 sec
  last saved crash : 0 days, 0 hrs, 0 min, 2 sec
  last saved hang : 0 days, 0 hrs, 0 min, 31 sec
cycle progress
  now processing : 735.1 (88.2%)
  runs timed out : 0 (0.00%)
stage progress
  now trying : havoc
  stage execs : 99/100 (99.00%)
  total execs : 180k
  exec speed : 1369/sec
fuzzing strategy yields
  bit flips : 8/32, 7/31, 6/29
  byte flips : 1/4, 0/3, 0/1
  arithmetics : 53/266, 0/280, 0/0
  known ints : 6/33, 4/104, 2/46
  dictionary : 0/0, 0/0, 0/0, 0/0
  havoc/splice : 754/170k, 0/0
  py/custom/rq : unused, unused, unused, unused
  trim/eff : 22.38%/1990, 75.00%
strategy: explore
state: started :-)
overall results
  cycles done : 0
  corpus count : 833
  saved crashes : 10
  saved hangs : 2
map coverage
  map density : 2.27% / 81.54%
  count coverage : 2.13 bits/tuple
findings in depth
  favored items : 451 (54.14%)
  new edges on : 599 (71.91%)
  total crashes : 22 (10 saved)
  total tmouts : 6 (0 saved)
item geometry
  levels : 5
  pending : 361
  pend fav : 0
  own finds : 832
  imported : 0
  stability : 100.00%
[cpu000: 25%]
```

Figure 6: AFL++ fuzzing 介面

9. 在 out/default/ 目錄下，crashes/ 目錄下會存放所有導致程式崩潰的輸入檔案，而 hangs/ 目錄下則會存放所有導致程式無限迴圈的輸入檔案。我可以透過 gdb 以以下指令來檢查崩潰的輸入檔案：

```
1 gdb --args ./fuzz-me out/default/crashes/<PoC filename>
```

4. (d) 漏洞分析

Vulnerability 1: Double Free

PoC crashes/id:000003,sig:06,src:000591,time:100031,execs:122095,op:havoc,rep:16 會導致程式觸發 double free 的漏洞。當我在 gdb 中配合 AddressSanitizer 執行該 PoC 時，會發現在 fuzz-me.c:6655:5 處觸發了 double free 的錯誤。

根據 gdb 的分析結果，可以發現在函數 mlxeotckigrz 當中，陣列 cghjfucyoytm 的內容在 fuzz-me.c:6641:51 中被宣告，並且在 fuzz-me.c:6652:9 當中，在進入 if 區塊中被釋放了第一次。然而出來到第 6655 行時，程式又無條件釋放了同一個陣列，導致 double free 的錯誤。

```
(gdb) r
Starting program: /home/b13901011/src/fuzz-me crashes/id:000003,sig:06,src:000591,time:100031,execs:122095,op:havoc,rep:16
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

==501520==ERROR: AddressSanitizer: attempting double-free on 0x616000001880 in thread T0:
[Detaching after fork from child process 501523]
#0 0x555555f8fb2 in free (/home/b13901011/src/fuzz-me+0xa4fb2) (BuildId: 2904b3ae46ed10ec10bf528c64c4c87055740fab)
#1 0x5555556cfb75 in m1xeotckigrz /home/b13901011/src/fuzz-me.c:6655:5
#2 0x5555556cf372 in vtqetpdrpobx /home/b13901011/src/fuzz-me.c:4806:9
#3 0x5555556b2e50 in gutmmsyccqox /home/b13901011/src/fuzz-me.c:5994:9
#4 0x5555556b2540 in lcazxhwiinez /home/b13901011/src/fuzz-me.c:3508:9
#5 0x5555556af7f0 in ntmiacljgcxv /home/b13901011/src/fuzz-me.c:6547:9
#6 0x5555556ae2 in ebnabpykxv /home/b13901011/src/fuzz-me.c:3376:9
#7 0x55555565c3b0 in axkznfedthw /home/b13901011/src/fuzz-me.c:4938:9
#8 0x55555565baa0 in vvwueljtjogu /home/b13901011/src/fuzz-me.c:1131:9
#9 0x5555556422d0 in lpopuvoycto /home/b13901011/src/fuzz-me.c:6635:9
#10 0x5555556df655 in main /home/b13901011/src/fuzz-me.c:7248:5
#11 0x7ffff7cf4249 in __libc_start_call_main csu/../sysdeps/nptl/libc_start_call_main.h:58:16
#12 0x7ffff7cf4304 in __libc_start_main csu/../csu/libc-start.c:360:3
#13 0x55555576410 in _start (/home/b13901011/src/fuzz-me+0x22410) (BuildId: 2904b3ae46ed10ec10bf528c64c4c87055740fab)

0x616000001880 is located 0 bytes inside of 514-byte region [0x616000001880,0x616000001a82)
freed by thread T0 here:
#0 0x555555f8fb2 in free (/home/b13901011/src/fuzz-me+0xa4fb2) (BuildId: 2904b3ae46ed10ec10bf528c64c4c87055740fab)
#1 0x5555556cfb69 in m1xeotckigrz /home/b13901011/src/fuzz-me.c:6652:9
#2 0x5555556cf372 in vtqetpdrpobx /home/b13901011/src/fuzz-me.c:4806:9
#3 0x5555556b2e50 in gutmmsyccqox /home/b13901011/src/fuzz-me.c:5994:9
#4 0x5555556b2540 in lcazxhwiinez /home/b13901011/src/fuzz-me.c:3508:9
#5 0x5555556af7f0 in ntmiacljgcxv /home/b13901011/src/fuzz-me.c:6547:9
#6 0x5555556ae2 in ebnabpykxv /home/b13901011/src/fuzz-me.c:3376:9
#7 0x55555565c3b0 in axkznfedthw /home/b13901011/src/fuzz-me.c:4938:9
#8 0x55555565baa0 in vvwueljtjogu /home/b13901011/src/fuzz-me.c:1131:9
#9 0x5555556422d0 in lpopuvoycto /home/b13901011/src/fuzz-me.c:6635:9
#10 0x5555556df655 in main /home/b13901011/src/fuzz-me.c:7248:5
#11 0x7ffff7cf4249 in __libc_start_call_main csu/../sysdeps/nptl/libc_start_call_main.h:58:16

previously allocated by thread T0 here:
#0 0x555555f925e in __interceptor_malloc (/home/b13901011/src/fuzz-me+0xa525e) (BuildId: 2904b3ae46ed10ec10bf528c64c4c87055740fab)
#1 0x5555556cf664 in m1xeotckigrz /home/b13901011/src/fuzz-me.c:6641:51
#2 0x5555556cf372 in vtqetpdrpobx /home/b13901011/src/fuzz-me.c:4806:9
#3 0x5555556b2e50 in gutmmsyccqox /home/b13901011/src/fuzz-me.c:5994:9
#4 0x5555556b2540 in lcazxhwiinez /home/b13901011/src/fuzz-me.c:3508:9
#5 0x5555556af7f0 in ntmiacljgcxv /home/b13901011/src/fuzz-me.c:6547:9
#6 0x5555556ae2 in ebnabpykxv /home/b13901011/src/fuzz-me.c:3376:9
#7 0x55555565c3b0 in axkznfedthw /home/b13901011/src/fuzz-me.c:4938:9
#8 0x55555565baa0 in vvwueljtjogu /home/b13901011/src/fuzz-me.c:1131:9
#9 0x5555556422d0 in lpopuvoycto /home/b13901011/src/fuzz-me.c:6635:9
#10 0x5555556df655 in main /home/b13901011/src/fuzz-me.c:7248:5
#11 0x7ffff7cf4249 in __libc_start_call_main csu/../sysdeps/nptl/libc_start_call_main.h:58:16

SUMMARY: AddressSanitizer: double-free (/home/b13901011/src/fuzz-me+0xa4fb2) (BuildId: 2904b3ae46ed10ec10bf528c64c4c87055740fab) in free
==501520==ABORTING
[Inferior 1 (process 501520) exited with code 01]
(gdb)
```

Figure 7: [Vulnerability 1] gdb 中的 double free 錯誤

Vulnerability 2: Stack Buffer Overflow

PoC crashes/id:000011,sig:06,src:000488,time:196159,execs:241438,op:havoc,rep:14 會導致程式 stack buffer overflow。當我在 gdb 中配合 AddressSanitizer 執行該 PoC 時，會發現 stack overflow 的觸發地點在 fuzz-me.c:3040:9 處。

我檢查該處的程式，可以發現第 3040 行位於 ngbaogfcgfv 函數當中，該行內容為透過 memcpy 進行記憶體複製，從 rvmltrxnnyedr 將長度 nhtdbsyeyipt 的記憶體複製到 qolxxudqisef。

```
1 memcpy(qolxxudqisef, rvmltrxnnyedr, nhtdbsyeyipt);
```

然而，根據第 3039 行可以看到 nhtdbsyeyipt 的值為 ≤ 510 的整數，而第 3029 行當中宣告

qolxxudqisef 的長度為 114 字元，因此當 nhtdbsyeyipt 的值大於 114 時，就會導致 stack buffer overflow。

```
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
=====
==1385==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7fffffffcd2 at pc 0x555555f863a bp 0x7fffffffcd30 sp 0x7fffffff500
WRITE of size 229 at 0x7fffffffcd2 thread T0
[Detaching after fork from child process 1388]
#0 0x555555f8639 in __asan_memcpy (/home/b13901011/src/fuzz-me+0xa4639) (BuildId: 2904b3ae46ed10ec10bf528c64c4c87055740fab)
#1 0x55555567be0f in ngbaogfcgfv /home/b13901011/src/fuzz-me.c:3040:9
#2 0x55555567b622 in zvfsjngfnveh /home/b13901011/src/fuzz-me.c:1879:9
#3 0x5555556dce62 in qkjhhqinyqdh /home/b13901011/src/fuzz-me.c:6104:9
#4 0x5555556cc622 in bpcsbhpgnvov /home/b13901011/src/fuzz-me.c:6236:9
#5 0x555555695700 in cgaraignonvg /home/b13901011/src/fuzz-me.c:4674:9
#6 0x555555694df2 in hjnxtchkcblj /home/b13901011/src/fuzz-me.c:2649:9
#7 0x5555556aaf72 in neqjohzqbbon /home/b13901011/src/fuzz-me.c:3266:9
#8 0x5555556db330 in fttkfdqkxkmv /home/b13901011/src/fuzz-me.c:5884:9
#9 0x5555556b5ba2 in xtmvbigmjezb /home/b13901011/src/fuzz-me.c:6481:9
#10 0x5555556df655 in main /home/b13901011/src/fuzz-me.c:7248:5
#11 0x7ffffcf4249 in __libc_start_call_main csu/../sysdeps/nptl/libc_start_call_main.h:58:16
#12 0x7ffffcf4304 in __libc_start_main csu/../csu/libc-start.c:360:3
#13 0x55555576410 in _start (/home/b13901011/src/fuzz-me+0x22410) (BuildId: 2904b3ae46ed10ec10bf528c64c4c87055740fab)

Address 0x7fffffffcd2 is located in stack of thread T0 at offset 146 in frame
#0 0x55555567b77f in ngbaogfcgfv /home/b13901011/src/fuzz-me.c:3028

This frame has 1 object(s):
[32, 146] 'qolxxudqisef' (line 3029) <== Memory access at offset 146 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork
(longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow (/home/b13901011/src/fuzz-me+0xa4639) (BuildId: 2904b3ae46ed10ec10bf528c64c4c87055740fab) in __asan_memcpy
Shadow bytes around the buggy address:
0x10007fff7960: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10007fff7970: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10007fff7980: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10007fff7990: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10007fff79a0: 00 00 00 00 00 00 00 00 00 f1 f1 f1 00 00 00 00
=>0x10007fff79b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10007fff79c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10007fff79d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10007fff79e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10007fff79f0: f1 f1 f1 f1 00 00 00 00 00 00 00 00 00 00 00 00
0x10007fff7a00: 00 00 02 f3 f3 f3 f3 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==1385==ABORTING
[Inferior 1 (process 1385) exited with code 01]
(gdb)
```

Figure 8: [Vulnerability 2] gdb 中的 stack buffer overflow 錯誤

Vulnerability 3: Segmentation fault

PoC crashes/id:000020,sig:11,src:000802,time:331908,execs:354578,op:havoc,rep:2 會導致程式觸發 segmentation fault。當我在 gdb 中執行該 PoC 時，會發現在 fuzz-me.c:3022 處，if (*kfdujeztzdrf > 0) 的條件判斷式中，因為指標 kfdujeztzdrf 存取到沒被分配的記憶體位置，導致程式觸發 segmentation fault。

我檢查該處的程式之前的程式，發現在第 3015 行當中，有一行 kfdujeztzdrf = 0；，這句可能是工程師筆誤，誤將指標直接指向 0 的位置，而不是改動他的數值。這導致在第 3022 行當中，if (*kfdujeztzdrf > 0) 的條件判斷式會存取到未被分配的記憶體位置，從而導致 segmentation fault。

我使用 gdb 逐行檢查時，透過在 fuzz-me.c:3015 處斷點，發現的確此處的賦值的確是導致 segmentation fault 而崩潰的原因。

```

Reading symbols from ./fuzz-me...
(gdb) r
Starting program: /home/b13901011/src/fuzz-me out/default/crashes/id:000020,sig:11,src:000802,time:331908,execs:354578,op:havoc,rep:2
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x00005555556a1dab in stpbzvzxvdc (kfdujeztzdrf=0x0, mijjoefvrhaa=0x603000000040 "\032", bxadhvmzptxg=32) at fuzz-me.c:3022
3022     if (*kfdujeztzdrf > 0) {

```

Figure 9: [Vulnerability 3] gdb 中的 segmentation fault 錯誤

```

Breakpoint 2, stpbzvzxvdc (kfdujeztzdrf=0x55555571ebe0 <dkcfjrfktxo>, mijjoefvrhaa=0x603000000040 "\032", bxadhvmzptxg=32)
at fuzz-me.c:3015
3015     kfdujeztzdrf = 0;
(gdb) n
3016     size_t kzxeknjmmhcf = ((uint32_t)lljfqmecmvqf[8]*35 + lljfqmecmvqf[10])^lljfqmecmvqf[5];
(gdb) n
3017     kzxeknjmmhcf = (kzxeknjmmhcf % 114) + 1;
(gdb) n
3018     memcpy(lcggijszpbvm, lljfqmecmvqf, kzxeknjmmhcf);
(gdb) n
3021     free(lljfqmecmvqf);
(gdb) n
3022     if (*kfdujeztzdrf > 0) {
(gdb) n

Program received signal SIGSEGV, Segmentation fault.
0x00005555556a1dab in stpbzvzxvdc (kfdujeztzdrf=0x0, mijjoefvrhaa=0x603000000040 "\032", bxadhvmzptxg=32) at fuzz-me.c:3022
3022     if (*kfdujeztzdrf > 0) {

```

Figure 10: [Vulnerability 3] 使用 gdb 逐行檢查的結果

6 貓物語（赤）

Reference:

- [Linear congruential generator \(LCG\) - Wikipedia](#)
- [LCG 隨機數破解](#)
- [Modular multiplicative inverse function in Python - Stack Overflow](#)
- <https://blog.csdn.net/PlutoZuo/article/details/134697641>

6. (a) 猜數字

Steps:

1. 首先，我觀察了 fatcat.py 的程式碼，發現它使用了一個線性同餘生成器（LCG）來產生隨機數。這個生成器的公式為：

$$X_{n+1} = (a \cdot X_n + c) \pmod{m}$$

2. 在程式碼當中我們已知 m ，但是並不知道 a 和 c 的值。然而，根據我搜尋的結果，LCG 演算法的 a 和 c 的值可以透過測量三次隨機數的差值來計算出來。過程如下：

$$\begin{aligned}
 X_1 &\equiv a X_0 + c \pmod{m}, \\
 X_2 &\equiv a X_1 + c \pmod{m}, \\
 X_2 - X_1 &\equiv a (X_1 - X_0) \pmod{m}
 \end{aligned}
 \implies
 \begin{aligned}
 a &\equiv \frac{X_2 - X_1}{X_1 - X_0} \pmod{m}, \\
 c &\equiv X_1 - a X_0 \pmod{m}.
 \end{aligned}$$

3. 我根據和伺服器互動三次，取得了 X_0 、 X_1 和 X_2 的值。接著，我透過 `solve_lcg.py` 程式碼計算出 a 和 c 的值。
4. 我撰寫了 `predict_lcg.py` 程式碼，並且在裡面透過 `pwntools` 套件來和伺服器互動，並且使用計算出來的 a 和 c 的值來預測下一個隨機數。猜中答案 100 次後，我取得足夠多信任值，並且成功獲得了 `flag`。

Flag:

NASA_HW11{pseudorandomness_does_not_guarantee_unpredictability}

6. (b) 考試答案卷

Steps:

1. 觀察程式，可以看到產生 `flag 2` 加密的程式如下：

```

1 def OTPEncrypt(msg: bytes) -> bytes:
2     key_len = 10
3     key = secrets.token_bytes(key_len)
4     enc = bytes(msg[i] ^ key[i % key_len] for i in range(len(msg)))
5     return enc
6
7 def send_exam_result():
8     hexlify(OTPEncrypt(FLAGS.encode())).decode()

```

這個加密方式是將一組長度為 10 的位元組陣列作為金鑰，重複循環的與明文逐位做 XOR 運算來加密。由於明文中一定包含前綴 `NASA_HW11{`，而密文 XOR 明文正好會還原出金鑰，因此可利用這段已知字串來破解整個金鑰。

2. 在我的 `decrypt_otp.py` 中，首先使用 `unhexlify` 將十六進位表示的密文還原為位元組，接著對所有可能的 `flag` 起始位置進行窮舉。對於每一個可能的起點，我們都假設明文是 `NASA_HW11{`，並且計算出對應的金鑰，加入候選金鑰列表中。
3. 隨後，我使用這些候選金鑰來解密整個密文，並透過正規表示式檢查解出的文字是否包含完整的 `flag`。雖然有兩組起始位置算出的明文都具有符合 `flag regex` 的片段，但其中一組明顯為雜亂亂碼，而另一組則是完整的 `flag` 格式。
4. 我解密出的文字是：

```

1 LinearAlgebraMidterm Name:Fatcat Score:86
  ↳ Flag:NASA_HW11{07p_k3y_mu57_b3_47_13457_45_10n6_45_pl41n73x7}

```

Flag:

NASA_HW11{07p_k3y_mu57_b3_47_13457_45_10n6_45_pl41n73x7}

6. (c) PoW

Steps:

1. 根據伺服器程式碼當中建立 proof-of-work 的方式，我需要找到一個數字 x ，使得 md5 的 0-8 位與給定的數字相同。
2. 然而可以發現，這個數字的範圍是 $[0, 2^{24})$ 的整數，因此我可以使用暴力建表的方式，預先計算出所有可能的 md5 前 8 位數字對應的數字 x 。這樣在伺服器要求我提供 proof-of-work 時，我只需要查表即可。
3. 我撰寫了 solve_pow.py 程式碼，首先用 python 的字典功能建立 md5 的對應表。當伺服器要求我提供 proof-of-work 時，我只需要查詢這個字典，找到對應的 x 即可。最後，我以足夠快的速度回應伺服器，得到每秒鐘 220 次回應的速度，並且成功獲得了 flag。

```
> python code/solve_md5.py
[+] Opening connection to 140.112.91.4 on port 1234: Done
[*] Switching to interactive mode
Wow! You can solve 220.64803882317165 PoWs per second!
Thanks for helping out with the work! Please accept this flag as thanks: NASA_HW11{https://youtu.be/1GxwDuV5JMc}

-----
| What's up? I'm Fatcat. Only my 麻吉 get to have my |
| flag! BTW, do you want to play a game wiwh me?   |
| [Current trust level: 0]                           |
| [Choices]:                                          |
| 1. Play a game                                     |
| 2. Ask for the flag                                |
| 3. Ask him about the exam result                   |
| 4. Help his 麻吉 do some work                      |
| 5. Ask him to verify your club membership          |
| 6. Ask him to prove his club membership            |
| 7. Leave                                            |
|-----|
Your choice: $ 7
Bye!
```

Figure 11: 獲得 flag

Flag:

NASA_HW11{https://youtu.be/1GxwDuV5JMc}

6. (d) Shared Key

因為所有人共用一組 shared key，因此我可以用 option 4 先取得 nonce，在開另一個視窗連線上之後，以 option 5 取得 SHA256(nonce||shared_key) 的值，在貼回上一個視窗即可得到 flag。

```
Your choice: 6
You don't believe I'm a member? Fine, I'll prove it.
Please give me a nonce: 3311738457
Proof: fatcat||b78873ba2cebaf260a0c3acc623a4837e3c7e2412440ec35817925b200d6d72c

Your choice: 5
Let me verify if you're a member first.
nonce: 3311738457
Please give me "your_name||SHA256(nonce||shared_key)": aaw||b78873ba2cebaf260a0c3acc623a4837e3c7e2412440ec35817925b200d6d72c
Wow, you play MapleStory too! Here is your flag: NASA_HW11{y0u_KN0w_r3F13C710n_4774cK}
```

Figure 12: 取得 nonce 和 shared key

Flag:

NASA_HW11{y0u_KN0w_r3F13C710n_4774cK}